

# 星上 DLM Sandwich-LoRA 微调实验步骤

Jetson Orin + Stable Diffusion x4 Upscaler + UC Merced 4× 遥感超分

## 1. 实验目标

本实验验证在上行链路窄、过顶时间有限、星上计算资源受限的条件下，是否可以在 Jetson Orin 上对遥感扩散大模型进行 LoRA-based 星上微调。

- 通信可行性：完整模型或完整参数增量为 GB 级，LoRA adapter 为 MB 到几十 MB 级。
- 计算可行性：冻结 DLM 主干，只训练 LoRA adapter，降低显存、训练时间和能耗。
- 方法假设：采用 Sandwich-LoRA，即微调浅层和深层，冻结中间层。浅层适配输入域偏移，深层适配输出细节，中间层保留通用扩散先验。

## 2. 第一版固定配置

| 项目   | 配置                                                                                  |
|------|-------------------------------------------------------------------------------------|
| 模型   | stabilityai/stable-diffusion-x4-upscaler                                            |
| 数据集  | UC Merced Land Use, 2100 张 256×256 RGB 遥感图像                                         |
| 任务   | 4× 超分辨率, 64×64 -> 256×256                                                           |
| 设备   | Jetson AGX Orin 32GB, 30 W 模式                                                       |
| 主方法  | Sandwich-LoRA : down_blocks[0] + up_blocks[-2:]                                     |
| 冻结层  | 其余 down blocks + mid block + early up blocks + VAE + text encoder                   |
| 训练参数 | rank=8, alpha=16, target=to_q/to_v, steps=200, batch=1, grad_accum=4, lr=1e-4, FP16 |
| 通信参数 | T_vis=600 s, R_up={0.128,0.5,1,5,10} Mbps, eta=0.7                                  |

## 3. Jetson Orin 环境部署

### 3.1 设置功耗模式

先查询 nvpmodel 支持的模式，再选择约 30 W 的功耗模式。不同设备的模式编号可能不同。

```
sudo nvpmodel -q
sudo nvpmodel -m 2
sudo jetson_clocks
```

### 3.2 安装依赖

PyTorch 和 torchvision 建议按 NVIDIA Jetson PyTorch 官方方式安装；其余依赖可用 pip 安装。

```
pip3 install --upgrade pip
pip3 install diffusers transformers accelerate peft safetensors pillow tqdm scikit-image datasets
huggingface_hub
huggingface-cli login
```

### 3.3 检查 CUDA

```
python3 - << 'EOF'
import torch
print("torch:", torch.__version__)
print("cuda:", torch.cuda.is_available())
print("device:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU")
EOF
```

## 4. 项目目录

```
mkdir -p sat_sandwich_lora
cd sat_sandwich_lora
mkdir -p data outputs
# save the training script as: onboard_sandwich_lora_sr.py
```

脚本文件名建议固定为 onboard\_sandwich\_lora\_sr.py。脚本需要包含四种模式：prepare\_ucmerced、comm、train、eval。

## 5. 数据集下载与准备

第一版使用 UC Merced。脚本通过 Hugging Face datasets 下载数据，并导出 train\_hr 和 val\_hr 两个目录。

```
python3 onboard_sandwich_lora_sr.py \
--mode prepare_ucmerced \
--data_root data/ucmerced \
--hr_size 256 \
--train_ratio 0.8
```

生成目录：

```
data/ucmerced/train_hr/
data/ucmerced/val_hr/
```

## 6. 通信可行性实验

该实验不训练模型，只计算不同参数更新量在给定上行速率和过顶窗口下的上传时间。

```
python3 onboard_sandwich_lora_sr.py \
--mode comm \
--update_sizes_mb 1200 80 30 15 7.5 \
--uplink_mbps 0.128 0.5 1 5 10 \
--contact_window_s 600 \
--eta 0.7 \
--output_csv outputs/comm_feasibility.csv
```

上传时间计算： $T_{up} = D_{update} / (\eta \times R_{up})$ 。其中  $R_{up}$  需从 Mbps 换算为 MB/s，即除以 8。

| 更新内容                    | 大小      | 解释                        |
|-------------------------|---------|---------------------------|
| Full model / full delta | 1200 MB | 完整模型或完整参数增量，通常无法在窄上行窗口内完成 |
| Large LoRA              | 80 MB   | 较大 adapter                |
| Medium LoRA             | 30 MB   | 中等 adapter                |
| Default LoRA            | 15 MB   | 默认 adapter                |
| Quantized LoRA          | 7.5 MB  | 量化 adapter 参考             |

## 7. Jetson 资源监控

训练前另开一个终端记录功耗、温度和内存使用。

```
mkdir -p outputs
tegrastats --interval 1000 | tee outputs/tegrastats_sandwich_train.log
```

训练结束后用 Ctrl+C 停止。论文中报告平均功耗、GPU/CPU 温度、内存使用峰值。

## 8. 主方法训练：Sandwich-LoRA

Sandwich-LoRA 微调 down\_blocks[0] 和最后两个 up blocks，冻结中间层。

```
python3 onboard_sandwich_lora_sr.py \
--mode train \
--train_dir data/ucmerced/train_hr \
--output_dir outputs/lora_sandwich_r8 \
--hr_size 256 \
```

```
--lr_size 64 \  
--rank 8 \  
--alpha 16 \  
--target qv \  
--lora_scope shallow_deep \  
--train_steps 200 \  
--batch_size 1 \  
--grad_accum 4 \  
--lr 1e-4 \  
--power_w 30
```

训练输出：

```
outputs/lora_sandwich_r8/pytorch_lora_weights.safetensors  
outputs/lora_sandwich_r8/train_log.csv  
outputs/lora_sandwich_r8/summary.csv
```

summary.csv 需要记录训练时间、估计能耗、峰值显存、adapter 大小、可训练参数量。

## 9. 模型评估

### 9.1 评估 Base DLM

```
python3 onboard_sandwich_lora_sr.py \  
--mode eval \  
--val_dir data/ucmerced/val_hr \  
--output_dir outputs/eval_base \  
--hr_size 256 \  
--lr_size 64 \  
--eval_max_images 20 \  
--num_inference_steps 25
```

### 9.2 评估 Sandwich-LoRA

```
python3 onboard_sandwich_lora_sr.py \  
--mode eval \  
--val_dir data/ucmerced/val_hr \  
--lora_dir outputs/lora_sandwich_r8 \  
--output_dir outputs/eval_sandwich_r8 \  
--hr_size 256 \  
--lr_size 64 \  
--eval_max_images 20 \  
--num_inference_steps 25
```

评估输出包括 eval\_metrics.csv、eval\_summary.csv 和若干可视化图像。

## 10. 对比实验

### 10.1 Last2-Up-LoRA

```
python3 onboard_sandwich_lora_sr.py \  
--mode train \  
--train_dir data/ucmerced/train_hr \  
--output_dir outputs/lora_last2up_r8 \  
--hr_size 256 \  
--lr_size 64 \  
--rank 8 \  
--alpha 16 \  
--target qv \  
--lora_scope last2_up \  
--train_steps 200 \  
--batch_size 1 \  
--grad_accum 4 \  
--lr 1e-4 \  
--power_w 30  
  
python3 onboard_sandwich_lora_sr.py \  
--mode eval \  
--val_dir data/ucmerced/val_hr \  
--lora_dir outputs/lora_last2up_r8 \  
--output_dir outputs/eval_last2up_r8 \  
--hr_size 256 \  
--lr_size 64 \  

```

```
--eval_max_images 20 \  
--num_inference_steps 25
```

## 10.2 Shallow-LoRA

```
python3 onboard_sandwich_lora_sr.py \  
--mode train \  
--train_dir data/ucmerced/train_hr \  
--output_dir outputs/lora_shallow_r8 \  
--hr_size 256 \  
--lr_size 64 \  
--rank 8 \  
--alpha 16 \  
--target qv \  
--lora_scope shallow \  
--train_steps 200 \  
--batch_size 1 \  
--grad_accum 4 \  
--lr 1e-4 \  
--power_w 30  
  
python3 onboard_sandwich_lora_sr.py \  
--mode eval \  
--val_dir data/ucmerced/val_hr \  
--lora_dir outputs/lora_shallow_r8 \  
--output_dir outputs/eval_shallow_r8 \  
--hr_size 256 \  
--lr_size 64 \  
--eval_max_images 20 \  
--num_inference_steps 25
```

## 10.3 All-LoRA 上界

```
python3 onboard_sandwich_lora_sr.py \  
--mode train \  
--train_dir data/ucmerced/train_hr \  
--output_dir outputs/lora_all_r8 \  
--hr_size 256 \  
--lr_size 64 \  
--rank 8 \  
--alpha 16 \  
--target qv \  
--lora_scope all \  
--train_steps 200 \  
--batch_size 1 \  
--grad_accum 4 \  
--lr 1e-4 \  
--power_w 30  
  
python3 onboard_sandwich_lora_sr.py \  
--mode eval \  
--val_dir data/ucmerced/val_hr \  
--lora_dir outputs/lora_all_r8 \  
--output_dir outputs/eval_all_r8 \  
--hr_size 256 \  
--lr_size 64 \  
--eval_max_images 20 \  
--num_inference_steps 25
```

## 11. 结果表与图

| 方法            | 微调位置                            | 需要报告的指标                    |
|---------------|---------------------------------|----------------------------|
| Base DLM      | 无 LoRA                          | PSNR, SSIM, 推理时间           |
| Shallow-LoRA  | down_blocks[0]                  | 质量、adapter size、训练时间、显存、能耗 |
| Last2-Up-LoRA | up_blocks[-2:]                  | 质量、adapter size、训练时间、显存、能耗 |
| Sandwich-LoRA | down_blocks[0] + up_blocks[-2:] | 主方法，报告全部指标                 |
| All-LoRA      | 所有 attention 层                  | 性能上界和资源开销上界                |

建议绘制：上传时间对比、PSNR/SSIM 对比、训练时间与能耗对比、adapter 大小与上传时间对比、综合效用对比。

## 12. OOM 降级顺序

- rank 从 8 降到 4。
- 训练步数从 200 降到 100。
- lora\_scope 从 shallow\_deep 改为 last2\_up。
- target 保持 qv，不要先使用 qkvo。
- 如果仍然不够，将 hr\_size/lr\_size 从 256/64 降到 128/32。

```
python3 onboard_sandwich_lora_sr.py \  
--mode train \  
--train_dir data/ucmerced/train_hr \  
--output_dir outputs/lora_sandwich_r4_light \  
--hr_size 256 \  
--lr_size 64 \  
--rank 4 \  
--alpha 8 \  
--target qv \  
--lora_scope shallow_deep \  
--train_steps 100 \  
--batch_size 1 \  
--grad_accum 4 \  
--lr 1e-4 \  
--power_w 30
```

## 13. 预期结论

- 完整 DLM 更新需要上传 GB 级参数，在窄上行和短过顶窗口下不可行。
- LoRA adapter 通常只有 MB 到几十 MB，可以在一次过顶窗口内上传。
- Sandwich-LoRA 比只微调浅层或只微调深层更完整：浅层处理输入域偏移，深层处理输出细节，中间层保留通用扩散先验。
- Sandwich-LoRA 应接近 All-LoRA 的图像质量，但训练时间、adapter size、能耗和上传时间更低。

第一版主配置可固定为：Jetson AGX Orin 32GB + Stable Diffusion x4 Upscaler + UC Merced 64→256 遥感超分 + LoRA rank=8 + Sandwich-LoRA + 200 training steps + 600 s 过顶窗口 + 1 Mbps 上行链路。